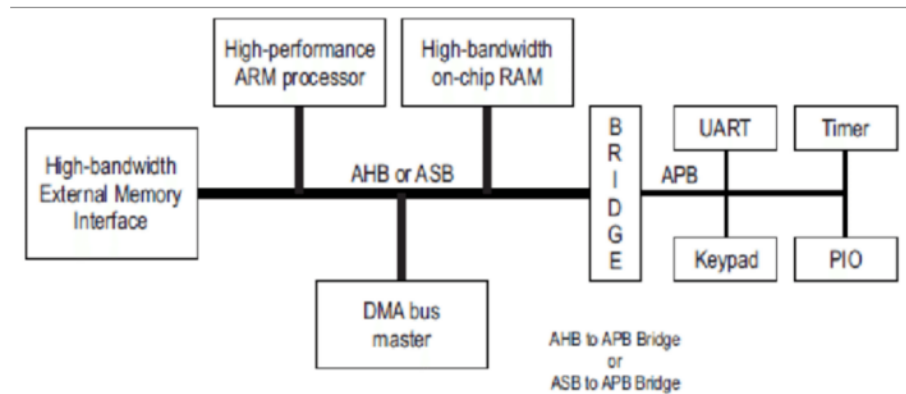


7)MINI PROJECT - AHB_APB BRIDGE

7.1)A typical AMBA-based Controller

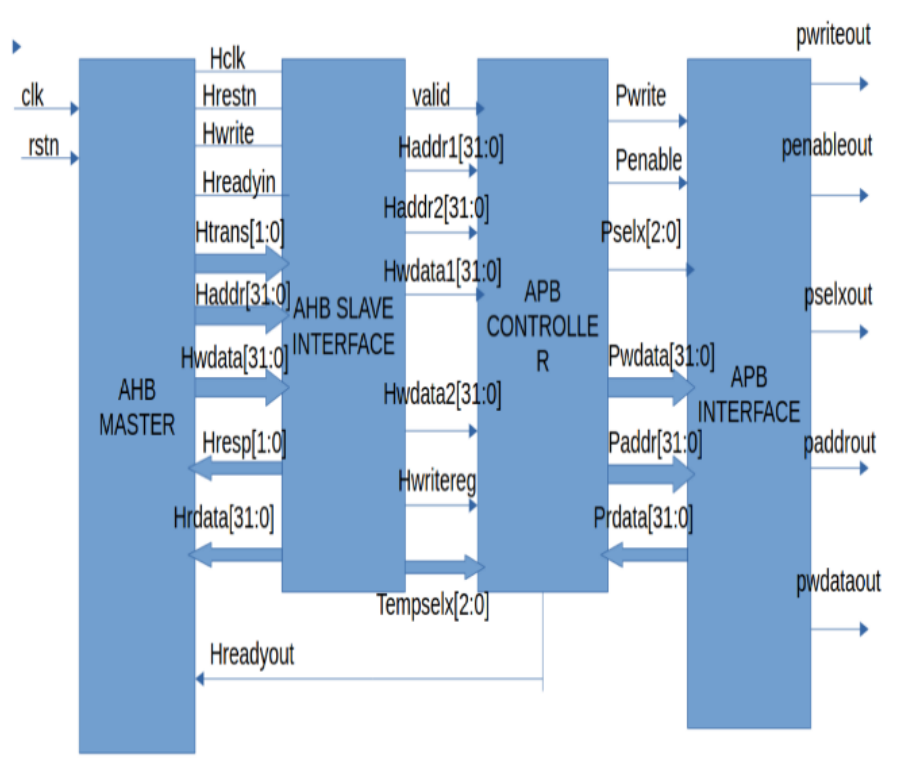


In an AMBA-based microcontroller, the internal communication between the CPU, memory, and peripherals is organized using different buses to balance performance and power efficiency. The block diagram shows this layered bus structure. At the center, we see the AHB (Advanced High-performance Bus) or ASB (Advanced System Bus), which acts as the main system bus. This bus interconnects the high-performance components such as an ARM Processor, which is a high-performance CPU core that executes instructions and communicates with memory and peripherals through the AHB; on-chip RAM, which is high-bandwidth RAM connected directly to AHB for fast instruction and data access; and an external memory interface, which allows the CPU to connect to external high-bandwidth memory devices like SDRAM or Flash.

A DMA Bus Master is a direct memory access controller that can transfer data between memory and peripherals directly over the AHB without involving the CPU, improving efficiency. Since many peripherals (like UART, Timer, GPIO, Keypad, etc.) do not require high-speed communication, the system uses a bridge (AHB-to-APB or ASB-to-APB). This bridge translates high-performance bus transactions into a simpler protocol suitable for the APB (Advanced Peripheral Bus). The APB is a low-power, low-complexity bus designed for peripheral devices. Peripherals connected here include UART (serial communication), Timer (timing

and counting functions), Keypad interface, and PIO (Programmable Input/Output). This separation ensures that performance-critical modules remain on the high-speed AHB/ASB, while simple peripherals operate on the APB to save power and reduce design complexity. The AHB/ASB is the main system bus connecting the CPU, high-speed memory, DMA, and external memory. The bridge converts AHB/ASB transactions into APB transactions. The APB is a low-power bus for simple, slower peripherals. High-performance modules (CPU, RAM, DMA) are on the AHB, while low-performance modules (UART, Timer, Keypad, PIO) are on the APB. This layered bus system makes AMBA-based microcontrollers scalable, modular, and efficient, balancing high performance for critical tasks with low power for peripheral operations.

7.2) Architecture Of AHB2APB Bridge



AMBA AHB and APB Signal Descriptions

In an AMBA-based microcontroller, communication between different

modules is achieved through standard bus signals defined in the AHB (Advanced High-Performance Bus) and APB (Advanced Peripheral Bus). Each bus uses a specific set of signals for control, data transfer, and synchronization.

On the AHB side, the key signals include:

- **HWDATA[31:0]**: The write data bus used by the master to transfer data to the selected slave during write operations. A 32-bit width is common, but it can be extended for higher bandwidth.
- **HSELx**: The slave select signal, generated by the address decoder, that identifies which slave is targeted for the current transfer.
- **HRDATA[31:0]**: The read data bus, driven by the slave during read operations, allowing the master to receive data.
- **HREADY**: A transfer status signal. When HIGH, it indicates the completion of a transfer. When LOW, it extends the transfer to allow the slave more time.
- **HRESP[1:0]**: The transfer response from the slave, which indicates whether the transaction is successful (OKAY) or needs special handling (ERROR, RETRY, SPLIT).

On the APB side, the signals are simpler, designed for low-power peripherals:

- **PCLK**: The APB clock, used to synchronize all transfers on the bus.
- **PRESET**: An active-low reset signal, normally connected directly to the system reset.
- **PADDR[31:0]**: The address bus, up to 32 bits wide, carrying the target peripheral address from the AHB-to-APB bridge.
- **PSELx**: Peripheral select signals, one for each slave, generated by the secondary decoder in the bridge to activate the chosen peripheral.

- **PENABLE**: The strobe signal that indicates the second phase of a transfer, ensuring correct timing on the APB.
- **PWRITE**: A direction control signal; HIGH for write operations and LOW for read operations.
- **PRDATA[31:0]**: The read data bus, driven by the selected peripheral during read cycles.
- **PWDATA[31:0]**: The write data bus, driven by the AHB-to-APB bridge during write cycles.

Together, these signals allow high-speed masters on the AHB to communicate seamlessly with low-power peripherals on the APB through the bridge. This separation of responsibilities ensures that performance-critical modules get high bandwidth while simple peripherals operate with minimal complexity and power consumption.

7.3) Approach used for Implementing the AHB to APB Bridge

The implementation of the AHB to APB bridge was carried out in multiple stages. At each step, the focus was on developing the RTL code, verifying its functionality through simulation, and checking synthesizability. The flow followed is described below:

1. AHB Slave Design

- The first stage involved writing the RTL code for the AHB slave module.
- A testbench was created to provide AHB master transactions (read and write operations).
- The design was simulated in ModelSim to observe waveform behavior and verify signal activity.
- The synthesis of the module was carried out in Quartus Prime, even though the initial simulations required multiple iterations to refine the logic.

2. APB Interface Design

- In the next stage, the APB module was coded in RTL, implementing the basic setup and access phases of the APB protocol.
- A separate testbench was written to test the APB signals (Psel, Penable, Pwrite) and data transfers.
- Simulations were done in ModelSim, and the results were analyzed for correctness. Adjustments were required during debugging to align with expected APB timing.
- The design was then taken through synthesis in Quartus Prime.

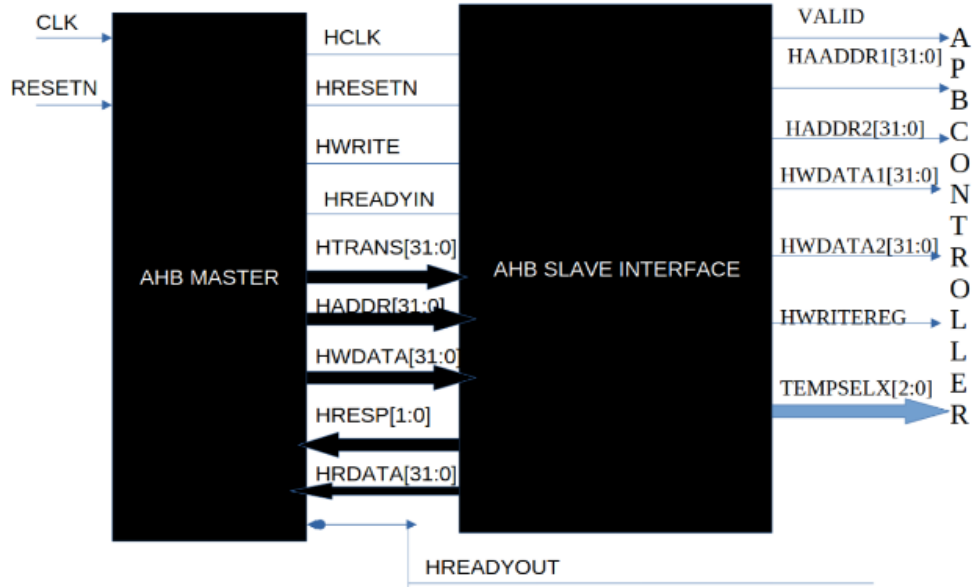
3. AHB to APB Bridge Integration

- After working on the individual modules, the bridge top module was created by connecting the AHB slave interface with the APB controller.
- At this stage, the objective was to verify that AHB transfers were being converted into valid APB operations.
- The first simulations were mainly used to check signal flow and debug mismatches in handshaking between the two bus protocols.

4. Bridge Verification and Synthesis

- A combined testbench for the bridge was developed to provide AHB transactions and observe their corresponding APB outputs.
- The simulation in ModelSim allowed checking of address, data, and control signal transitions at both interfaces.
- Finally, the complete bridge design was synthesized using Quartus Prime to confirm resource mapping and timing feasibility.

7.4) AHB Slave Block, Simulation Waveform and Synthesis



AHB Slave Interface

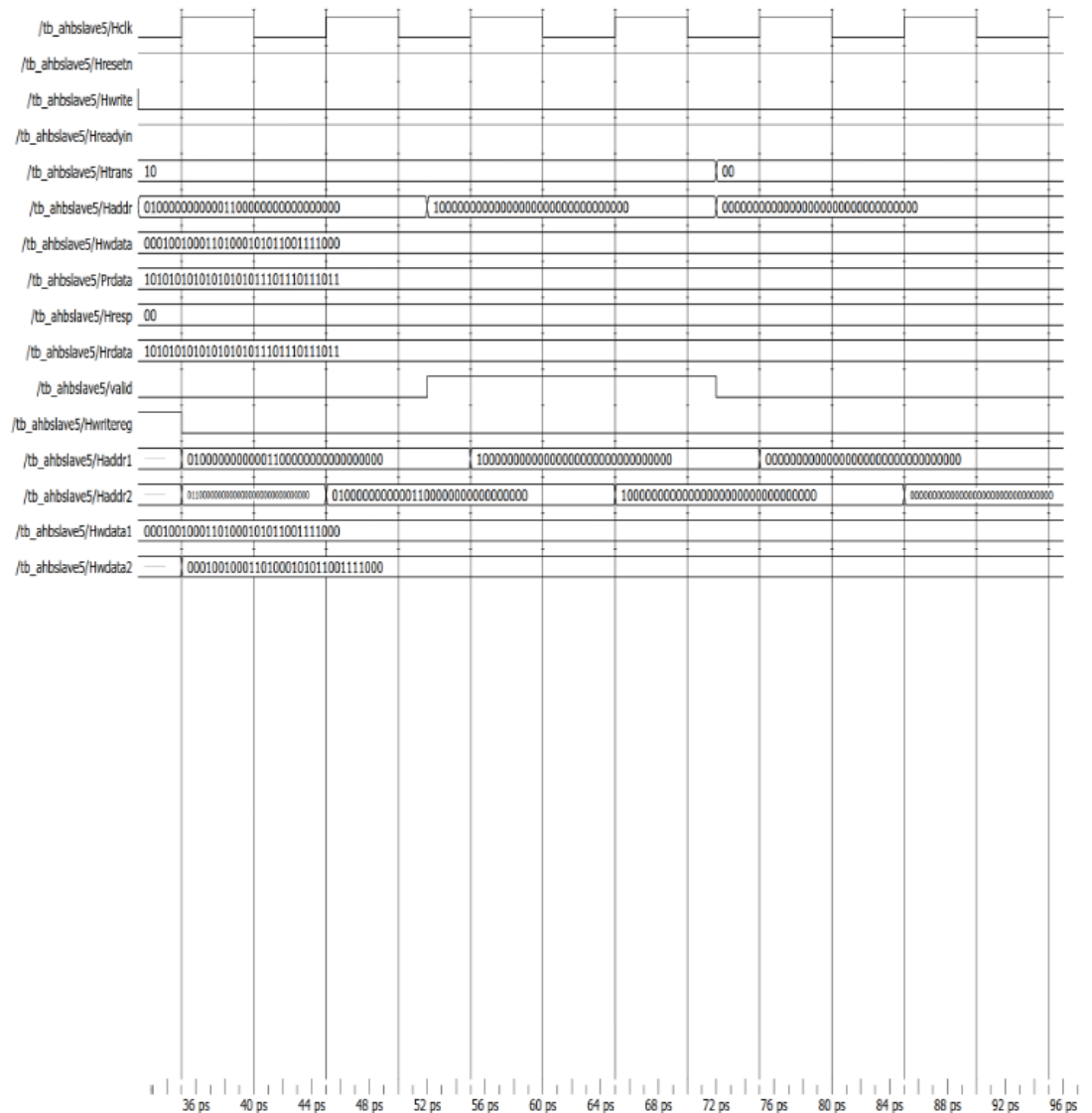
The AHB Slave Interface is a key block in the AHB to APB bridge architecture, responsible for receiving transactions from the AHB master and preparing them for transfer to the APB controller. It acts as a protocol translator at the AHB side, ensuring that the incoming high-performance AHB transactions are correctly interpreted, buffered, and synchronized before being passed further into the bridge.

Functional Role

- The AHB slave monitors control signals such as HWRITE, HTRANS, HREADYIN, and HRESETN to determine the type and validity of transactions.
- The address phase is handled through HADDR[31:0], which specifies the target peripheral or memory location.
- During write operations, data is received through HWDATA[31:0] and stored in temporary registers for forwarding to the APB side.

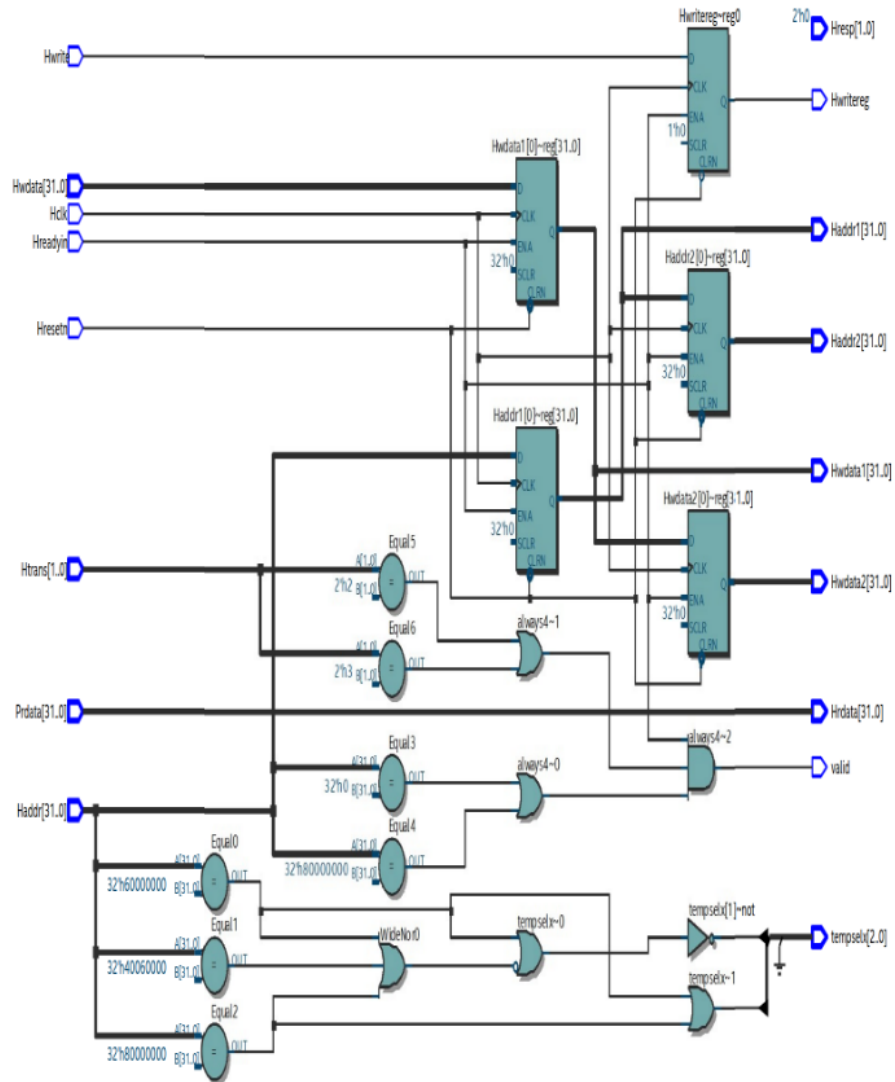
- For read operations, the slave retrieves data from the APB interface and drives it back to the master via HRDATA[31:0].
- The response mechanism is supported using HRESP[1:0] and HREADY-OUT, which provide status feedback to the master, indicating whether a transfer is complete, needs to wait, or has encountered an error.

WAVEFORM

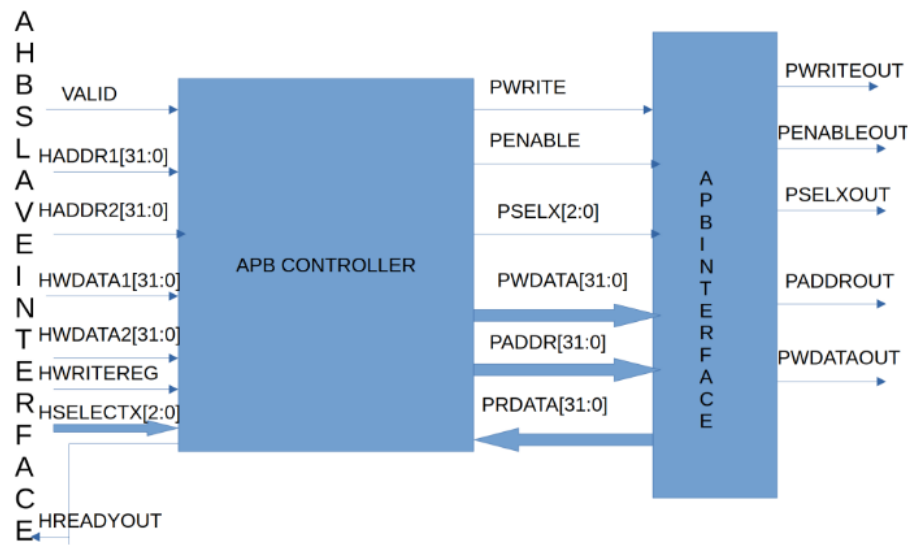


Entity:tb_ahbslave5 Architecture: Date: Sun Aug 10 22:39:53 IST 2025 Row: 1 Page: 1

SYNTHESIS



7.5) APB controller Block, Simulation Waveform and Synthesis



The block diagram illustrates the core elements and communication flow within an APB (Advanced Peripheral Bus) system, a key component in embedded system design.

APB Controller

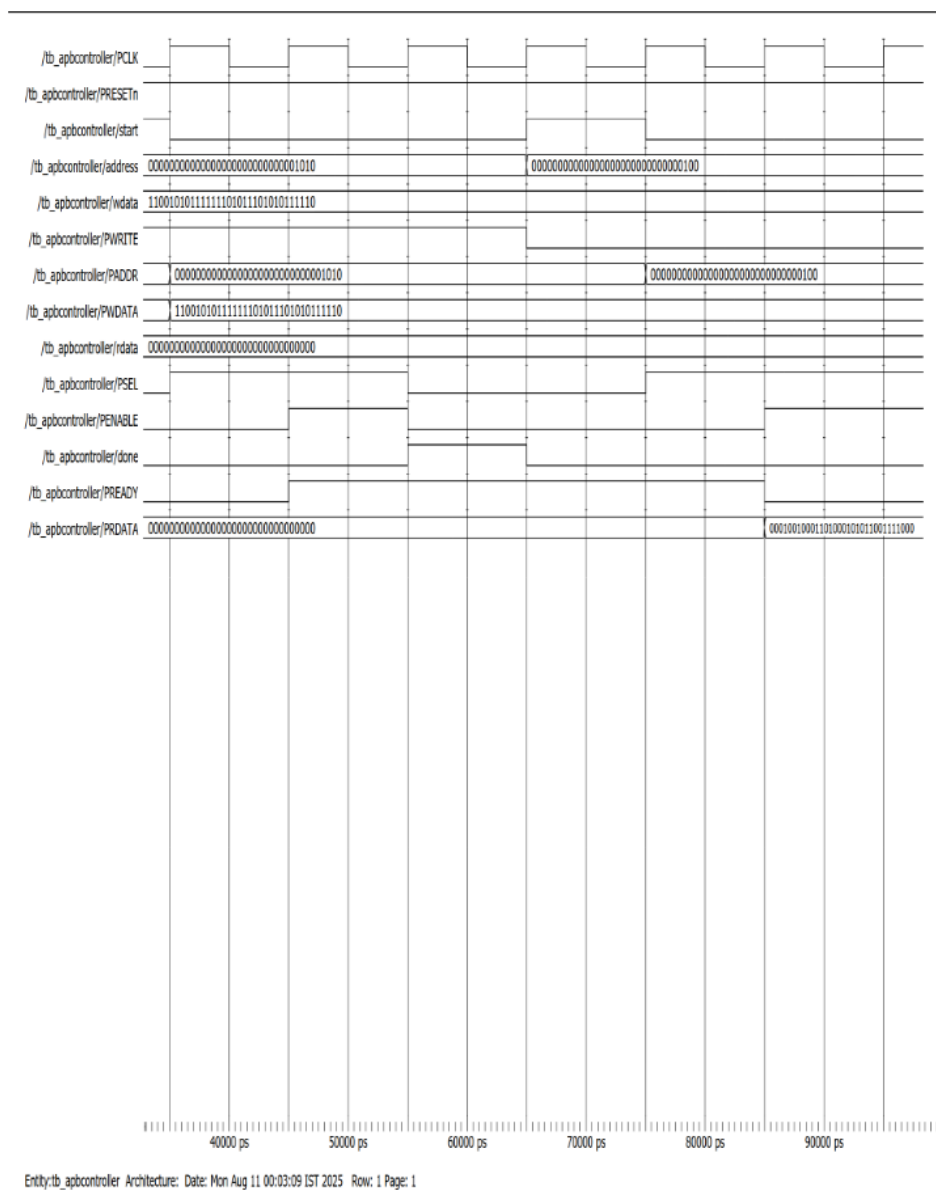
The APB Controller is a crucial block in the AHB to APB bridge architecture, responsible for accepting transactions from the AHB slave interface and managing the conversion to the APB protocol. It coordinates the organization of address, control, and data signals required to interface with APB-compliant peripherals, ensuring the timing and handshaking for each operation match the APB specification.

Functional Role

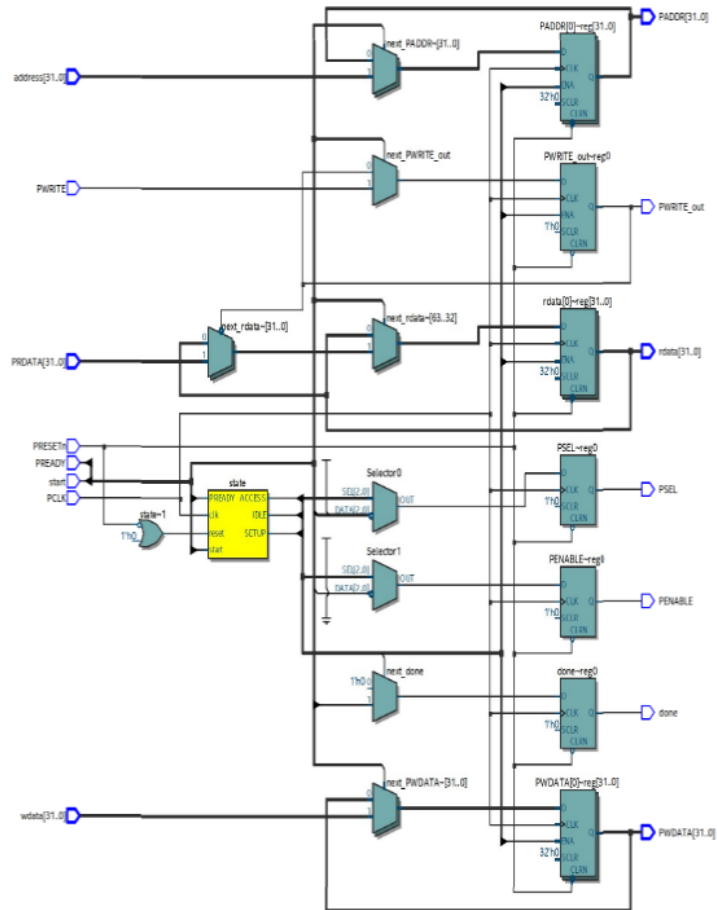
- The APB controller monitors control signals such as PWRITE, PSEL, PENABLE, and PRESETN to determine the type and phase of ongoing APB transactions.
- The address phase is managed using PADDR[31:0], directing transactions to the appropriate APB peripheral.

- During write operations, data forwarded from the AHB side is placed on PWDATA[31:0] and sent to the peripheral.
- For read operations, the controller receives data from the APB peripheral via PRDATA[31:0] and makes it available to the AHB interface.
- Status and protocol management are handled with PENABLE and peripheral device feedback, completing APB protocol handshakes and synchronizing timing with the rest of the bridge.

WAVEFORM



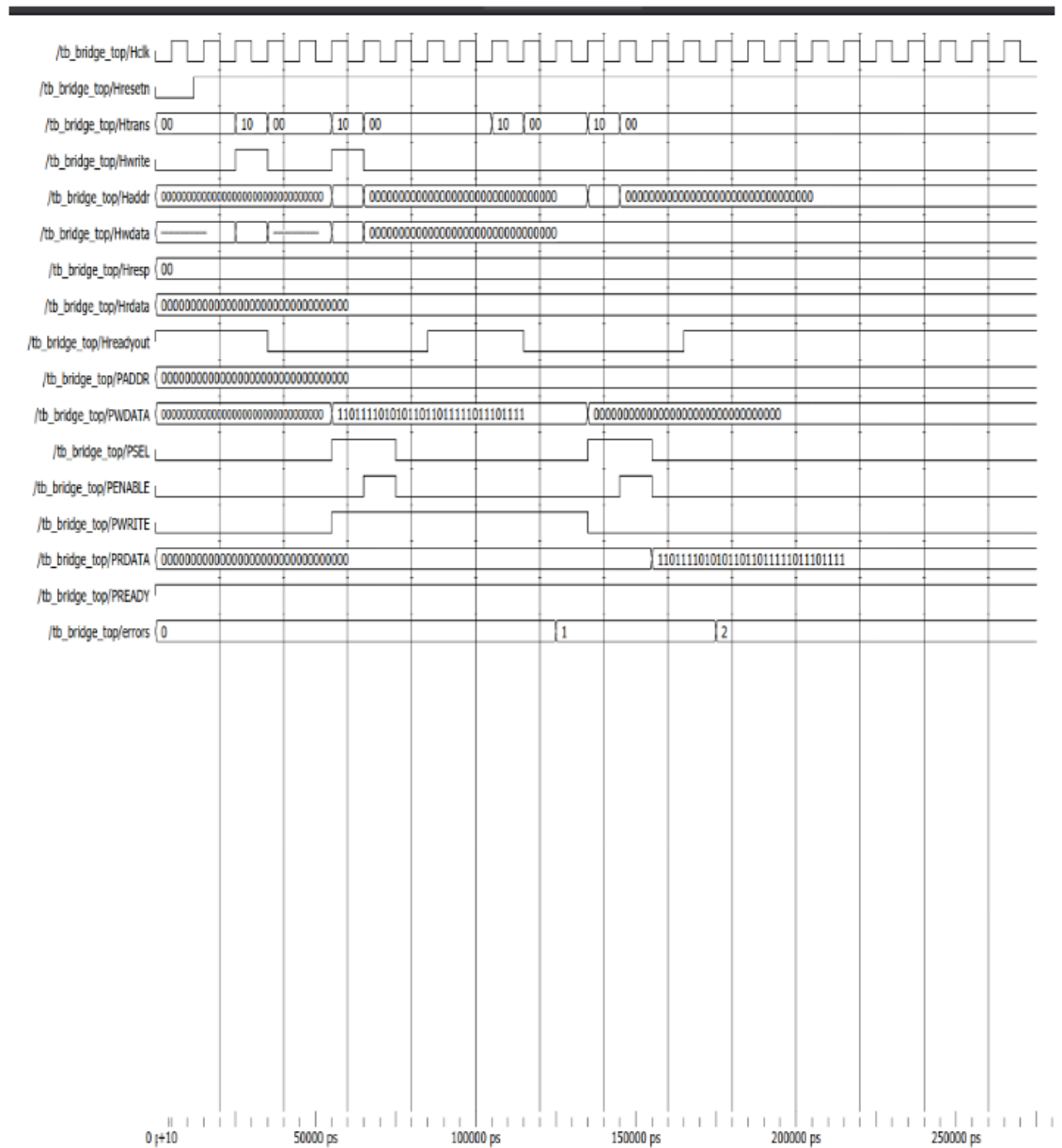
SYNTHESIS



7.6) BRIDGE TOP SIMULATION AND SYNTHESIS

The Bridge Top module integrates the AHB slave and the APB controller to enable smooth communication between the high-performance AHB bus and the low-power APB bus. The AHB slave latches address, data, and control signals from the AHB master, while the APB controller translates these into proper APB protocol signals (PADDR, PWDATA, PSEL, PENABLE, PWRITE) for the peripheral. During a write, data from the AHB master is passed through to the APB slave, and during a read, the data returned from the APB slave (PRDATA) is forwarded to the AHB master (Hrdata). The bridge ensures correct sequencing using a finite state machine (IDLE $\rightarrow$ SETUP $\rightarrow$ ACCESS) on the APB side. Simulation in ModelSim verifies the signal handshaking and functionality, while synthesis in Quartus Prime confirms hardware implementability. The expected waveform shows valid address and data transfers with proper PSEL/PENABLE toggling and acknowledgment of read/write completion.

WAVEFORM



Entity:tb_bridge_top Architecture: Date: Mon Aug 11 01:49:18 IST 2025 Row: 1 Page: 1

SYNTHESIS

